

## 03-02 编码规范与开发约定

系统名称：智慧能源管理系统 系统版本：V4.6.0 文档版本：V1.0 编写日期：2026-09-14 适用对象：全体后端 / 前端 / 全栈研发 强制级别：标注【强制】的条目 CodeReview 不予通过；标注【建议】的为推荐实践

---

### 目录

1. 总则与工程结构
  2. 后端分层规范
  3. 命名规范
  4. REST 接口与统一返回约定
  5. 参数校验、异常处理与日志
  6. Sa-Token 鉴权与数据权限
  7. MyBatis-Plus 与多数据源使用规范
  8. 事务与幂等
  9. Redis / Redisson / Lock4j
  10. MQ 消费规范（手动 ACK / 重试 / 死信）
  11. EasyExcel 导入导出
  12. 接口文档（Swagger / Javadoc）
  13. 单元测试规范
  14. 前端开发规范
  15. Git 分支、提交与 CodeReview
  16. 代码生成器使用
  17. 安全编码
  18. 加性开发铁律
- 

### 1. 总则与工程结构

#### 1.1 技术基线（不得随意升级）

- JDK **1.8**；Spring Boot **2.7.9**；Maven 多模块；
- 持久层：MyBatis-Plus 3.5.3.1 + dynamic-datasource 3.5.2 + p6spy；
- 鉴权：Sa-Token 1.34.0（含 sa-token-jwt）；

- 消息：Spring AMQP + RabbitMQ、Eclipse Paho MQTT;
- 缓存/锁：Redis、Redisson 3.20.0、Lock4j 2.2.3;
- 定时任务：XXL-Job 2.3.1 + Spring @Scheduled;
- 工具：Hutool 5.8.15、fastjson2 2.0.53、EasyExcel 3.2.1、Lombok 1.18.36;
- 前端主用工程：Vue 2.6.12 + Element UI 2.15.12 + Vue CLI 4 (zhurong-admin-ui)。

升级任何基线依赖必须提方案、全量回归，严禁个人私自升级。

## 1.2 模块职责边界

| 模块                         | 允许的内容                                                               | 禁止的内容                       |
|----------------------------|---------------------------------------------------------------------|-----------------------------|
| zhurong-ems-admin          | 启动类、Controller、启动相关配置                                               | 写业务 SQL、直接操作 Mapper 之外的业务逻辑 |
| zhurong-ems-system         | domain(entity/bo/vo/dto/enums)、service、mapper、XML、MQ/MQTT/定时任务等业务实现 | 定义 Web Controller           |
| zhurong-ems-framework      | 安全、拦截器、AOP、全局异常、过滤器、配置                                              | 写具体业务                       |
| zhurong-ems-common         | 通用返回、基类、工具、注解、常量、枚举、异常                                              | 依赖业务模块                      |
| zhurong-ems-job            | XXL-Job 执行器装配; JobHandler 可放 system 模块                              | —                           |
| zhurong-ems-oss/sms/extend | 对象存储、短信、调度中心/监控独立服务                                                 | 反向依赖业务模块                    |

## 1.3 两大根包约定

- com.cpems：启动与 Web 层。Controller 统一在 com.cpems.web.controller.<域>，例如：
  - com.cpems.web.controller.hospital.HospitalDeviceController
  - com.cpems.web.controller.system.SyncEngineController
  - com.cpems.web.controller.report.ReportEngineController
- com.ruoyi：业务与框架层。业务主体在 com.ruoyi.system.\*，医院子域统一在 com.ruoyi.system.hospital.\*。

启动类 cpEmsApplication.java 已配置 @ComponentScan(basePackages = {"com.cpems", "com.ruoyi"}), 新增包必须落在这两个根包之下。

Mapper 扫描与 XML 位置在 application.yml 全局配置, 新增模块不要再写 @MapperScan:

mybatis-plus:

```
mapperPackage: com.cpems.**.mapper,com.ruoyi.**.mapper,com.autoee.**.mapper
mapperLocations: classpath*:mapper/**/*.xml
typeAliasesPackage: com.cpems.**.domain,com.ruoyi.**.domain,com.autoee.**.domain
```

---

## 2. 后端分层规范

### 2.1 标准分层与调用方向

Controller (admin)

↓ BO / 查询参数

Service 接口 IXxxService (system)

↓

XxxServiceImpl (system)

↓

XxxMapper (system, interface) + resources/mapper/\*\*/\*.xml

↓

MySQL(master) / TDengine(@DS("td"))

- 【强制】 Controller 不直接调 Mapper (仪表盘联表聚合等特殊场景除外, 需注释说明) ;
- 【强制】 跨模块业务通过 Service 接口调用, 不允许 Controller 之间互调;
- 【强制】 ServiceImpl 中 Mapper 字段统一命名 baseMapper, 使用构造注入:

@RequiredArgsConstructor

@Service

```
public class HospitalDeviceServiceImpl implements IHospitalDeviceService {
```

```
    private final HospitalDeviceMapper baseMapper;
    private final IHospitalDataScopeService dataScopeService;
    // ...
```

```
}
```

参考实现: HospitalDeviceServiceImpl.java。

2.2 domain 四件套 + enums

医院模块的包结构是全项目推荐范式（com.ruoyi.system.hospital）：

| 包            | 类型                   | 用途                   | 示例                                        |
|--------------|----------------------|----------------------|-------------------------------------------|
| domain       | 实体 Entity            | 与表一一对应，@TableName    | HospitalDevice、HospitalDeviceData         |
| domain.enums | 枚举                   | 状态、类型等有限值            | HospitalDeviceType、HospitalMetricDataType |
| bo           | 入参对象 Business Object | 接收前端查询/新增/修改参数，带校验注解 | HospitalDeviceBo                          |
| vo           | 出参对象 View Object     | 返回前端/页面组装            | HospitalDeviceVo、HospitalDeviceRealtimeVo |
| dto          | 传输对象                 | 跨系统/跨层报文             | IotCallbackRequest（外部 IOT 回调报文）           |
| constant     | 常量                   | 模块内常量                | HospitalConstants                         |
| energy       | 专题 VO                | 大屏/能源分析专用出参          | HospitalEnergyOverviewVo                  |

约束：

- 【强制】Entity 不直接暴露给前端做入参，新增/修改接口一律用 BO；出参用 VO，避免敏感字段与表结构泄露；
- 【强制】类注释必须包含中文用途说明与 @author cpems；
- 【建议】BO→Entity、Entity→VO 使用 BeanUtil/BeanCopyUtils，转换集中在 ServiceImpl。

3. 命名规范

3.1 类命名

| 类型 | 规则 | 示例 |
|----|----|----|
|----|----|----|

| 类型            | 规则                               | 示例                                                  |
|---------------|----------------------------------|-----------------------------------------------------|
| Controller    | XxxController                    | HospitalDeviceController、<br>ReportEngineController |
| Service 接口    | IXxxService（业务子域<br>沿用此风格）       | IHospitalDeviceService<br>、ISyncEngineService       |
| Service 实现    | XxxServiceImpl                   | HospitalDeviceServiceImpl                           |
| Mapper        | XxxMapper                        | HospitalDeviceMapper、<br>EnergyMapper               |
| 实体            | 业务名（名词）                          | HospitalAlarmRule                                   |
| BO / VO / DTO | XxxBo / XxxVo / XxxDto           | HospitalAlarmRuleBo                                 |
| 枚举            | XxxType / XxxStatus /<br>XxxEnum | TopicType、<br>HospitalDeviceType                    |
| 配置类           | XxxConfig /<br>XxxConfiguration  | HospitalRabbitConfig、<br>MqttConfiguration          |
| 常量类           | XxxConstants                     | HospitalConstants                                   |
| 异常            | XxxException                     | ServiceException                                    |
| 注解            | Xxx 或 XxxAnnotation              | @Log、@RepeatSubmit                                  |

### 3.2 方法 / 变量 / 常量

- 方法：小驼峰，动词开头：queryPageList、insertByBo、updateByBo、deleteWithValidByIds、selectVoById、buildMenus；
- 布尔方法/变量：is/has/should 前缀：isUnrestricted()、signEnabled；
- 常量：全大写下划线：RETRY\_QUEUE、MAX\_RETRY\_TIMES、ALARM\_RULE\_THRESHOLD；
- 抽象/不推荐直接使用的方法加 @Deprecated 并注释替代方案；
- 严禁魔法值：业务码、状态字符串（如 "0" 正常、"2" 删除）必须走枚举或常量。

### 3.3 包命名

- 全小写，单数形式：controller.hospital、service.impl、domain.enums；
- 新业务域在 com.ruoyi.system.<域> 与 com.cpems.web.controller.<域> 下对称建包。

### 3.4 数据库表与字段

- 表名：小写下划线，模块前缀：医院模块统一 hospital\_ 前缀（hospital\_device、hospital\_alarm\_record）；系统表沿用 sys\_ 前缀；
  - 字段：小写下划线，自动驼峰映射已开启（mapUnderscoreToCamelCase: true）；
  - 主键：id (BIGINT, 雪花)；业务编号字段 xxx\_code；外键引用 xxx\_id；
  - 审计字段：create\_by/create\_time/update\_by/update\_time/remark；
  - 逻辑删除：del\_flag, 0 存在 / 2 删除；
  - 状态字段：status, 0 正常 / 1 停用；开关类用 char(1)；
  - TDengine 列名避开保留字（如 value 用 metric\_value, 参考 hospital\_init\_tdengine.sql）；
  - 表/字段必须有中文 COMMENT，表字符集 utf8mb4。
- 

## 4. REST 接口与统一返回约定

### 4.1 路径与动词【强制】

系统管理类接口前缀 /system/xxx；业务域按域前缀（/hospital/xxx、/report/xxx、/system/integration/sync/...）。标准动作：

| 动作   | HTTP 方法 | 路径示例                                     |
|------|---------|------------------------------------------|
| 分页列表 | GET     | /hospital/device/list                    |
| 详情   | GET     | /hospital/device/info/{id}（部分老模块为 /{id}） |
| 新增   | POST    | /hospital/device                         |
| 修改   | PUT     | /hospital/device                         |
| 删除   | DELETE  | /hospital/device/{ids}（支持批量，逗号/数组）       |
| 导出   | POST    | /hospital/device/export                  |

参考：HospitalDeviceController.java。

### 4.2 统一返回体

普通返回统一使用 R.java：

```
public class R<T> {  
    public static final int SUCCESS = 200;  
    public static final int FAIL = 500;  
    private int code;
```

```

        private String msg;
        private T data;
    }

    return R.ok();           // 操作成功
    return R.ok(vo);         // 带数据
    return R.fail("设备编号已存在");

```

分页返回统一使用 `TableDataInfo<T>`，由 Service 组装：

```

LambdaQueryWrapper<HospitalDevice> lqw = buildQueryWrapper(bo);
Page<HospitalDeviceVo> result = baseMapper.selectVoPage(pageQuery.build(), lqw);
return TableDataInfo.build(result);

```

入参分页统一对象 `PageQuery` (`com.ruoyi.common.core.domain.PageQuery`)，Controller 方法签名直接声明即可接收 `pageNum/pageSize`。

Controller 继承 `BaseController`，增删改结果用 `toAjax(rows)`；不要自行 new `HashMap` 拼返回。

### 4.3 请求体与 Content-Type

- 新增/修改：@RequestBody XxxBo, Content-Type: application/json;
- 查询/导出：表单参数或 URL Query, BO 直接作为方法参数（不加 @RequestBody）；
- 时间格式统一 yyyy-MM-dd HH:mm:ss (Jackson 已全局配置)。

---

## 5. 参数校验、异常处理与日志

### 5.1 参数校验

- 【强制】BO 字段使用 JSR-303 注解：@NotBlank、@NotNull、@NotEmpty、@Size、@Pattern 等；
- 【强制】新增/修改分组校验：@Validated(AddGroup.class) / @Validated(EditGroup.class)（包 com.ruoyi.common.core.validate）；
- 【强制】Controller 类上标注 @Validated, PathVariable 校验如 @NotNull(message="主键不能为空") @PathVariable Long id;
- 校验消息中文、具体，例如 "设备名称不能为空"，并支持国际化 key 时写 {xxx.not.blank}。

## 5.2 异常处理

- 业务失败统一抛 `ServiceException` (`com.ruoyi.common.exception.ServiceException`)，由 framework 全局异常处理器转成 `R.fail`；
- **【禁止】** `catch` 住异常后吞掉只 `e.printStackTrace()`；捕获必须记录日志或重新抛出；
- **【强制】** MQ 消费等链路内异常要区分「可重试」与「不可重试」，不可恢复异常落错误日志/失败表，避免无限重投；
- 参数非法由框架统一处理，Controller 内不手写 `try/catch` 包校验。

## 5.3 日志

- 使用 SLF4J: `private static final Logger log = LoggerFactory.getLogger(Xxx.class);` 或 `Lombok @Slf4j`;
  - 占位符写法: `log.info("[IOT 回调] 处理成功 requestId={}" points={}", msgId, points.size());`，**【禁止】** 字符串拼接打日志；
  - 链路日志统一中文方括号前缀，如 `[IOT 回调]`、`[医院报警]`、`[开始]`-接收到 RabbitMQ 的消息，便于检索；
  - `error` 必须带异常对象: `log.error("落库失败", e);`
  - 敏感信息 (token、密码、身份证、手机号全量) **【禁止】** 打印；
  - 操作审计使用 `@Log(title="医院设备", businessType=BusinessType.INSERT)` 注解 (见 6.2)，不要手写操作日志表；
  - 本地日志配置 `classpath:logback.xml`，控制台文件 `./logs/sys-console.log` (`actuator logfile` 指向该文件)。
- 

## 6. Sa-Token 鉴权与数据权限

### 6.1 登录与认证

- 登录入口 `SysLoginController.java`: `POST /login (@SaIgnore 免登录) → SysLoginService.login(username,password,code,uuid)` 完成验证码校验、BCrypt 比对、Sa-Token 登录签发；
- Token 请求头: `Authorization: Bearer <token>` (`sa-token.token-prefix=Bearer`，从 header 读取)；
- 免登录路径在 `application.yml` 的 `security.excludes`，医院回调 `/hospital/callback/**` 在排除列表中 (但有自定义 X-IOT-Token 鉴权)；

- 获取当前用户：LoginHelper.getLoginUser() / LoginHelper.getUserId()。

## 6.2 权限注解【强制】

每个业务接口必须挂权限码（除明确的公开接口）：

```
@SaCheckPermission("hospital:device:list")    // 单权限
@SaCheckPermission(value = {"hospital:device:add","hospital:device:edit"}, mode = SaMode.OR)
@SaCheckRole("admin")                        // 角色校验
@SaIgnore                                    // 跳过登录（谨慎使用，需评审）
```

权限码命名：模块:实体:动作，动作固定

list/query/add/edit/remove/export/import。权限码必须与 sys\_menu.perms 中的菜单/按钮配置一一对应（医院模块见 数据库脚本/hospital\_menu\*.sql）。

写操作必须加操作日志与防重复提交：

```
@SaCheckPermission("hospital:device:add")
@Log(title = "医院设备", businessType = BusinessType.INSERT)
@RepeatSubmit()
@PostMapping()
public R<Void> add(@Validated(AddGroup.class) @RequestBody HospitalDeviceBo bo) {
    return toAjax(service.insertByBo(bo));
}
```

## 6.3 数据权限

- 平台级数据权限走部门/角色体系（sys\_role.data\_scope 等），Service 中通过数据权限服务（ISysDataScopeService）解析可见范围；
- 医院院区数据权限范式：HospitalDataScopeService.isUnrestricted() 判断是否全量、resolveAccessibleAreas() 解析当前用户可见院区集合，内部委托平台 ISysDataScopeService，业务查询把可见集合作为 Wrapper 条件；
- 【强制】列表/统计 SQL 必须带入数据范围条件，不得只靠前端隐藏菜单。

---

## 7. MyBatis-Plus 与多数据源使用规范

### 7.1 实体注解

```
@Data
@TableName("hospital_device")
public class HospitalDevice {
```

```

@TableId(value = "id")           // 全局 id-type=ASSIGN_ID, 雪花 ID
private Long id;

@TableField("device_name")      // 列名不一致时显式指定
private String deviceName;

@TableLogic                      // 逻辑删除: 0 存在 2 删除
private String delFlag;
}

```

- 【强制】主键策略全局 ASSIGN\_ID（雪花），不要用数据库自增；
- 【强制】有删除需求的业务表必须带 del\_flag 并在实体加 @TableLogic，删除走 deleteById 逻辑删除；
- 全局配置已设 insert/update 策略 NOT\_NULL：为 null 的字段不参与更新，「清空字段」需显式置空串或用 UpdateWrapper；
- 时序实体例外：TDengine 超表实体（如 Energy）无逻辑删除、无雪花主键，首列必须是 ts TIMESTAMP。

## 7.2 Wrapper 与分页【强制】

- 一律使用 LambdaQueryWrapper，禁止字符串列名硬编码：

```

LambdaQueryWrapper<HospitalDevice> lqw = Wrappers.lambdaQuery();
lqw.like(StringUtils.isNotBlank(bo.getDeviceName()), HospitalDevice::getDeviceName, bo.getDeviceName());
lqw.eq(StringUtils.isNotBlank(bo.getDeviceType()), HospitalDevice::getDeviceType, bo.getDeviceType());
lqw.orderByDesc(HospitalDevice::getCreateTime);

```

- 条件构造统一封装为 ServiceImpl 的 buildQueryWrapper(bo) 私有方法；
- 分页：baseMapper.selectVoPage(pageQuery.build(), lqw)，返回 TableDataInfo.build(page)；列表用 selectVoList，详情用 selectVoById（均来自 BaseMapperPlus，接口签名 BaseMapperPlus<M, T, V>）。

## 7.3 XML 编写

- XML 放在 zhurong-ems-system/src/main/resources/mapper/...，命名 XxxMapper.xml（扩展 XML 可用 XxxMapperExtend.xml，会被 \*Mapper\*.xml 通配加载）；
- 简单 CRUD 用 BaseMapperPlus，不写冗余 XML；仅复杂联表/聚合/批量写 XML；

- 【强制】参数全部走 #{}，【禁止】用 \${} 拼接用户输入（动态表名场景除外，且必须做白名单/正则清洗，参考 HospitalDeviceDataConsumer.generateTableName 的 [^a-zA-Z0-9\_] 清洗）；
- 复用 <sql> 片段，避免 SELECT \*。

## 7.4 多数据源与 TDengine

- 数据源由 dynamic-datasource 管理，默认源 master（MySQL autoee\_ems）；
- TDengine Mapper/方法标注 @DS("td")，如 EnergyMapper、HospitalTdMapper；
- TD 写入总开关：spring.datasource.dynamic.datasource.td.enabled（本地启动参数显式 =false）。业务中注入开关：

```
@Value("${spring.datasource.dynamic.datasource.td.enabled:false}")
private boolean tdDbEnabled;
```

- 【强制】TDengine 写失败不得影响 MySQL 主链路：try/catch 包住、仅记日志（见 HospitalDeviceDataConsumer.saveToTdengine）；
- TDengine SQL 注意：自动建子表语法 insert into 子表 using 超表 tags(...) values(...); 列名避开保留字；子表名只允许字母数字下划线。

## 8. 事务与幂等

- 需要原子性的多写操作加 @Transactional(rollbackFor = Exception.class)，默认只回滚 RuntimeException 的坑要规避；
- 事务注解加在 Service 的 public 方法；事务方法内避免 RPC/MQ 发送等不可回滚操作，正确顺序是「先落库、后发消息」；
- 写接口默认加 @RepeatSubmit()（基于间隔内相同 URL+body 判定），前端 request 拦截器也有 1 秒防重复提交兜底；
- 外部回调天然可能重推：以 msgId 做幂等（写入 hospital\_callback\_log.request\_id），消费侧以设备+指标+时间戳做唯一性兜底；
- 批量操作使用 insertBatch / rewriteBatchedStatements=true（连接串已开启）提升性能，注意超大批量分片。

## 9. Redis / Redisson / Lock4j

- 工具类优先 RedisUtils (common 模块, 封装 String/Hash/Set/过期时间); key 命名 业务:实体:标识, 如 hospital:device:online:{deviceId}, 必须设置合理 TTL;
  - 本地共享 Redis 使用 db 13, 且有密码 difyai123456, Spring 与 Redisson 配置必须同时带密码 (application-local.yml 已示范 redisson.singleServerConfig);
  - 分布式锁使用 Lock4j (lock4j-redisson-spring-boot-starter): 方法上加 @Lock4j(keys="#bo.deviceCode", expire=30000, acquireTimeout=3000);
  - 防超卖/并发更新用 Redisson 锁或数据库乐观锁, 禁止「先查后写」无锁并发;
  - 【禁止】把 Redis 当唯一持久存储; 缓存必须可重建。
- 

## 10. MQ 消费规范 (手动 ACK / 重试 / 死信)

### 10.1 通用规则

- 队列、交换机、绑定必须用 @Bean 在 Config 中声明, 不得只在管理台手工创建;
- 生产发送使用 rabbitTemplate.convertAndSend(exchange, routingKey, payload); 新链路使用 JSON 字符串 (医院链路), 老设备链路为 Java 序列化 Map;
- 【强制】消费使用手动 ACK (AcknowledgeMode.MANUAL / ackMode = "MANUAL");
- 成功 channel.basicAck(deliveryTag, false); 失败按是否可恢复决定 basicNack(tag, false, requeue);
- 【禁止】不可恢复异常 (脏数据) 反复 requeue=true 造成毒消息风暴。

### 10.2 项目两条消费范式

1. 老设备链路: SimpleMessageListenerContainer  
(MessageListenerConfig.java) + ChannelAwareMessageListener  
(MyAckReceiver), 并发数 1;
2. 医院链路 (推荐范式): 注解驱动 + 死信重试队列
  - 主队列 hospital.device.data.queue 声明 x-dead-letter-exchange / x-dead-letter-routing-key;

- 主消费者 HospitalDeviceDataConsumer: 成功 ack; 失败 `basicNack(tag,false,false)` 进死信;
- 重试消费者 HospitalDeviceDataRetryConsumer: 从重试队列读取消息头 `x-retry-count`, 重新投递主队列, 最多 `MAX_RETRY_TIMES=3` 次, 超限 ack 并 error 日志转人工;
- 新业务一律采用「主队列 + DLQ 重试 + 次数头」模式。

### 10.3 消费幂等与隔离

- 消费逻辑必须幂等;
- 不同业务域使用独立交换机/队列 (医院链路 `hospital.topic.exchange` 与老设备 `EquipmentTopicExchange` 物理隔离), 不得混用;
- 消费内调用第三方 (邮件、工单) 失败要可降级, 不阻断 ACK 主流程。

---

## 11. EasyExcel 导入导出

- 版本 3.2.1 (`com.alibaba.excel`);
- 导出走 `ExcelUtil.exportExcel(list, sheetName, Entity.class, response)` (参考 `SysPostController.export`), 导出接口路径 `POST /export`, 权限 `xxx:export`, `@Log(businessType=BusinessType.EXPORT)`;
- 实体字段使用注解 `@ExcelProperty("列名")`, 不需要导出的字段加 `@ExcelIgnore`; 类上可用 `@ExcelIgnoreUnannotated` (只导出显式注解字段, 见 `Energy`);
- 大数据量导入使用 EasyExcel 的 `ReadListener` 流式读取 + 分批入库 (建议每 500/1000 条一批), 不要一次性 `readAll`;
- 导入必须做: 表头校验、字段校验、错误行收集并返回错误报告。

---

## 12. 接口文档 (Swagger / Javadoc)

- 文档组件 `springdoc-openapi 1.6.15 + therapi-runtime-javadoc-scribe 0.15.0` (根 pom 已注册注解处理器): **Javadoc 注释即接口文档**;
- 【强制】Controller 类、每个接口方法、BO/VO 字段必须写中文 Javadoc, 含 `@param/@return`;
- 分组配置在 `application.yml`: 1.演示模块 `com.cpems.demo`、2.系统模块 `com.cpems.web`、3.代码生成模块 `com.cpems.generator`; 新 Controller 落在 `com.cpems.web` 下自动入文档;
- 调试授权: Swagger UI 点 Authorize 填 Bearer <token> (与请求头一致)。

---

## 13. 单元测试规范

### 13.1 框架与执行机制

- JUnit 5 (Jupiter) ; 根 pom 的 surefire 2.22.2 配置:

```
<groups>${profiles.active}</groups>          <!-- local/dev/prod -->
<excludedGroups>exclude</excludedGroups>
```

即: 测试方法/类用 @Tag("local")、@Tag("dev")、@Tag("prod") 标注在哪个环境的构建中执行; 明确不跑的用 @Tag("exclude")。

### 13.2 命名与编写约定

- 测试类与源码同包, 放在各模块 src/test/java, 命名 XxxTest / XxxServiceImplTest;
- 方法名表达场景: should\_returnPageList\_when\_queryByDeviceType() 或中文下划线 分页查询\_按设备类型\_返回匹配记录;
- 结构 Given-When-Then; 断言使用 JUnit5 Assertions / AssertJ, 【禁止】无断言测试;
- Service 单测使用 Mockito (@ExtendWith(MockitoExtension.class)) mock Mapper 与外部调用;
- 纯工具/转换逻辑测试标 @Tag("local"), 保证本地 mvn test 可跑; 依赖中间件的集成测试标 @Tag("dev") 或 exclude;
- 【建议】新增核心 Service 方法单测覆盖率  $\geq 70\%$ , 报警规则匹配、报文解析、签名校验等高风险逻辑必须有用例 (正常/边界/异常三类)。

---

## 14. 前端开发规范

### 14.1 目录与技术约定 (zhurong-admin-ui)

- 框架: Vue 2.6 Options API 为主 + Element UI 2.15.12 + Vuex 3 + vue-router 3 + vue-i18n 8 + sass;
- 关键目录:

```
src/
├─ api/           # 接口封装, 按后端域分子目录 (hospital/、system/、report/...)
├─ views/         # 页面, 与菜单 component 字符串对应 (@/views/xxx)
├─ components/    # 全局/业务组件 (Pagination、RightToolbar、DictTag、FileUpload...)
```

|                  |                                               |
|------------------|-----------------------------------------------|
| — store/modules/ | # Vuex: user、permission、app、dict              |
| — router/        | # constantRoutes / dynamicRoutes              |
| — utils/         | # request.js、auth.js、ruoyi.js、字典 Dict         |
| — plugins/       | # this.\$modal/\$tab/\$auth/\$download 等全局挂载  |
| — directive/     | # v-hasPermi、v-hasRole、v-clipboard、dialogDrag |
| — layout/        | # 框架布局                                        |
| — i18n/          | # zh-CN / en / ru / id                        |
| — permission.js  | # 路由守卫 (登录态 → GetInfo → GenerateRoutes)       |

## 14.2 API 封装【强制】

- 所有请求走 @/utils/request，禁止页面直接 import axios；
- 一个后端实体一个 api 文件，导出

listXxx/getXxx/addXxx/updateXxx/delXxx，参考 api/hospital/device.js：

```
import request from '@/utils/request'
export function listDevice(query) {
  return request({ url: '/hospital/device/list', method: 'get', params:
    query })
}
export function addDevice(data) {
  return request({ url: '/hospital/device', method: 'post', data })
}
```

- request 拦截器自动注入 Authorization: Bearer <token> 与 Content-Language: zh\_CN，并做 1 秒重复提交拦截；401 自动弹重新登录；blob/arraybuffer 直接返回用于下载。

## 14.3 模板语法【强制·高频坑】

- 属性绑定一律 v-bind，**严禁属性内 mustache**：:yName="\$t('xxx')", yName="{{ \$t('xxx') }}"（编译失败并阻断整包构建）；
- 提交前自查：rg '='>{{' src -g '\*.vue'；
- v-for 必须带 :key；v-if 与 v-for 不同元素；多分支用计算属性。

## 14.4 权限控制

- 按钮/链接级权限：指令 v-hasPermi="['hospital:device:add']"、角色 v-hasRole（注册于 src/directive/index.js，实现在 directive/permission/）；
- JS 内判断使用 this.\$auth.hasPermi('xxx') / hasRole（plugins/auth.js）；
- 菜单与路由权限由后端 /getRouters 下发，前端 store/modules/permission.js 的 GenerateRoutes 把 component 字符串映射到 @/views/\*\*（Layout/ParentView/InnerLink 特殊处理）；新增页面必须同时在菜单表配置 component 路径。

## 14.5 字典、Vuex、国际化、组件与样式

- 字典：Options API 使用 `dicts: ['sys_normal_disable']` 选项（DictData 全局 mixin 自动加载到 `dict.type.xxx`），展示用全局组件 `<dict-tag :options="dict.type.xxx" :value="row.status"/>`；JS 取标签用 `selectDictLabel`；命令式获取 `this.getDicts('xxx')`；
- Vuex：用户态 `store/modules/user.js`（Login/GetInfo/LogOut），路由态 `permission.js`；跨页共享状态才进 Vuex，页面私有状态留 `data`；
- 国际化：文案必须走 `$t('key')`（`src/i18n/lang/zh-CN.js` 等 4 语言），新增 `key` 至少补 `zh-CN/en`；菜单名称国际化方案见 `docs/后台菜单国际化详细处理方案.md`；
- 组件：优先复用全局组件（Pagination 分页、RightToolbar 表格右侧按钮、FileUpload/ImageUpload、Editor）；弹窗内表单校验走 `el-form rules`；
- 样式：`scoped + sass`；公共样式放 `assets/styles/`（`ruoyi.scss/theme.scss`）；禁止内联大片样式；主题色/间距走变量；
- ECharts 统一 5.4，深浅主题使用 `assets/echartsTheme/`；数字滚动用 `vue-count-to`；
- 代码风格：ESLint（`.eslintrc.js`）+ `lint-staged` 提交自动 `fix`；提交前跑 `npm run lint`；
- 大屏/3D（`three 0.153`）相关页面独立目录，注意销毁时释放 WebGL 资源；医院大屏 `/hospital/screen` 为独立全屏路由，已在 `permission.js` 特殊处理，勿注入 Layout。

---

## 15. Git 分支、提交与 CodeReview

### 15.1 分支模型

- `master/main`：与生产一致，只接受合并请求合入；
- `develop`（或当前主干开发分支）：集成分支；
- `feature/<模块>-<简述>`：新功能，如 `feature/hospital-alarm-dedup`；
- `hotfix/<日期/问题>`：生产缺陷修复；
- `fix/<简述>`：日常缺陷。

### 15.2 提交信息格式

`<type>(<scope>): <subject>`

`<body>` 可选

type 固定: feat (新功能)、fix (缺陷)、perf (性能)、refactor、docs、test、chore、style、build、ci、revert。scope 用模块名: hospital、mqtt、auth、report、ui 等。

示例:

feat(hospital): IOT 回调增加 HMAC-SHA256 签名校验

fix(mqtt): electric/all 数组报文缺失 createTime 时自动补时间

docs(03): 新增开发环境搭建手册

- 一次提交只做一件事, 禁止「功能+格式化+ unrelated 修复」混提;
- 提交前本地编译通过 (后端 mvn clean package -DskipTests、前端 npm run lint) ;
- 不提交本地配置改动 (如 application-local.yml 个人口令、IDE 文件、target/dist/node\_modules) 。

### 15.3 CodeReview 清单

- ☐ 是否符合分层与包结构; 权限码、@Log、@RepeatSubmit 是否齐全;
- ☐ 逻辑删除/雪花 ID/数据权限是否遗漏;
- ☐ SQL 是否有注入风险、是否 SELECT \*、大表是否走索引;
- ☐ MQ 消费 ACK/重试/幂等是否正确; TD 失败是否降级;
- ☐ 异常是否被吞、日志是否合规、敏感信息是否脱敏;
- ☐ 前端是否走 request 封装、是否有属性 mustache、按钮权限是否补齐;
- ☐ 是否加性变更, 有无改动既有功能/生产脚本;
- ☐ 菜单/权限标识/字典/国际化是否同步配置。

---

## 16. 代码生成器使用

- 工程内存在 zhurong-ems-generator 模块 (Velocity 模板) 与前端页面 src/views/tool/gen、src/api/tool/gen.js, 但根 pom 的 <modules> 中该模块当前被注释、默认不参与构建;
- 需要使用时: 放开根 pom 中 zhurong-ems-generator 依赖与模块注释, 在生成器页面配置表名、包路径、模块名 (建议按本规范的 com.ruoyi.system.<域> + com.cpems.web.controller.<域> 生成), 生成 entity/bo/vo/mapper/xml/service/controller 与前端 api/views 后再人工补齐:
  - @SaCheckPermission 权限码与 sys\_menu 菜单 SQL;
  - @TableLogic、校验分组、数据权限;

- 前端字典与国际化 key;
  - 生成代码只是脚手架，必须经过本规范清单整改后再提交；当前未启用生成器时，按 03-03 文档第 11 章的手工脚手架步骤执行。
- 

## 17. 安全编码

- **口令存储**：用户密码使用 BCrypt（工程内 TestBCrypt.java 可辅助生成校验串），禁止明文/MD5/可逆加密存储；
  - **敏感字段**：手机号、身份证等按需加解密/脱敏展示；mybatis-encryptor 默认关闭且默认算法仅 BASE64（非加密），如需启用必须配置真实对称密钥（AES/SM4，bouncycastle 已引入），不得把 BASE64 当加密；
  - **SQL 注入**：全部参数化 #{ }；动态排序字段/表名做白名单；
  - **XSS**：公共 xss.enabled=true 已对 /system/\*,/monitor/\*,/tool/\* 过滤；富文本走白名单；前端渲染富文本用受限组件（Editor/quill），不要 v-html 未清洗内容；
  - **越权**：每个接口鉴权 + 数据权限双重控制；医院回调类免登录接口必须自定义 Token/IP/签名校验（参考 IotCallbackAuthServiceImpl: X-IOT-Token 常量时间比对、IP 白名单、可选 X-IOT-Sign = hex(HMAC\_SHA256(secret, timestamp + "\n" + msgId))) ；
  - **文件上传**：走 OSS 模块，校验类型/大小（multipart 全局 10MB/20MB），文件名重命名，禁止上传可执行文件到 Web 可访问目录；
  - **接口安全**：对外暴露接口默认最小权限；错误信息不返回堆栈与内部细节；
  - **依赖漏洞**：关注 snakeyaml 等被根 pom 锁定修复版本的依赖，禁止回退；
  - **配置安全**：密钥不入库，生产走环境变量/配置中心；本地示例配置不得带生产口令。
- 

## 18. 加性开发铁律

医院智慧能源等新模块均按「叠加、不改旧」原则建设，后续开发必须延续：

1. **【强制】不删除、不破坏原有功能**：老接口签名、老字段语义、老 MQTT/MQ 主题保持兼容；确需变更必须保留兼容期并双写/适配；
2. **【强制】新能力以新表/新类/新模块叠加**：医院模块全部使用 hospital\_ 前缀新表（hospital\_init.sql 中显式声明「加性开发，不影响原有表」），业务代码集中在 com.ruoyi.system.hospital 与 com.cpems.web.controller.hospital；

3. 【强制】生产 **compose / 部署脚本不动**: docker-compose.full.yml、docker-compose-54.yml 等生产文件严禁为本地调试修改; 本地差异通过 application-local.yml profile 隔离;
4. 【强制】**新链路 with 老链路隔离**: 医院 MQ 使用独立 hospital.topic.exchange, 不与 EquipmentTopicExchange 混用;
5. 【强制】**可开关的非核心能力默认不影响主链路**: TDengine 写入、报警评估、邮件通知失败均降级处理;
6. 【建议】新功能优先复用平台能力 (菜单权限、字典、OSS、短信、操作日志、数据权限), 不另起炉灶;
7. 【建议】数据库变更同时提供「新建脚本」与「回滚说明」, 初始化脚本统一放 数据库脚本/, 命名带模块/迭代前缀。

---

配套文档: 03-01 开发环境搭建手册.md | 03-03 模块详细设计与实现说明.md